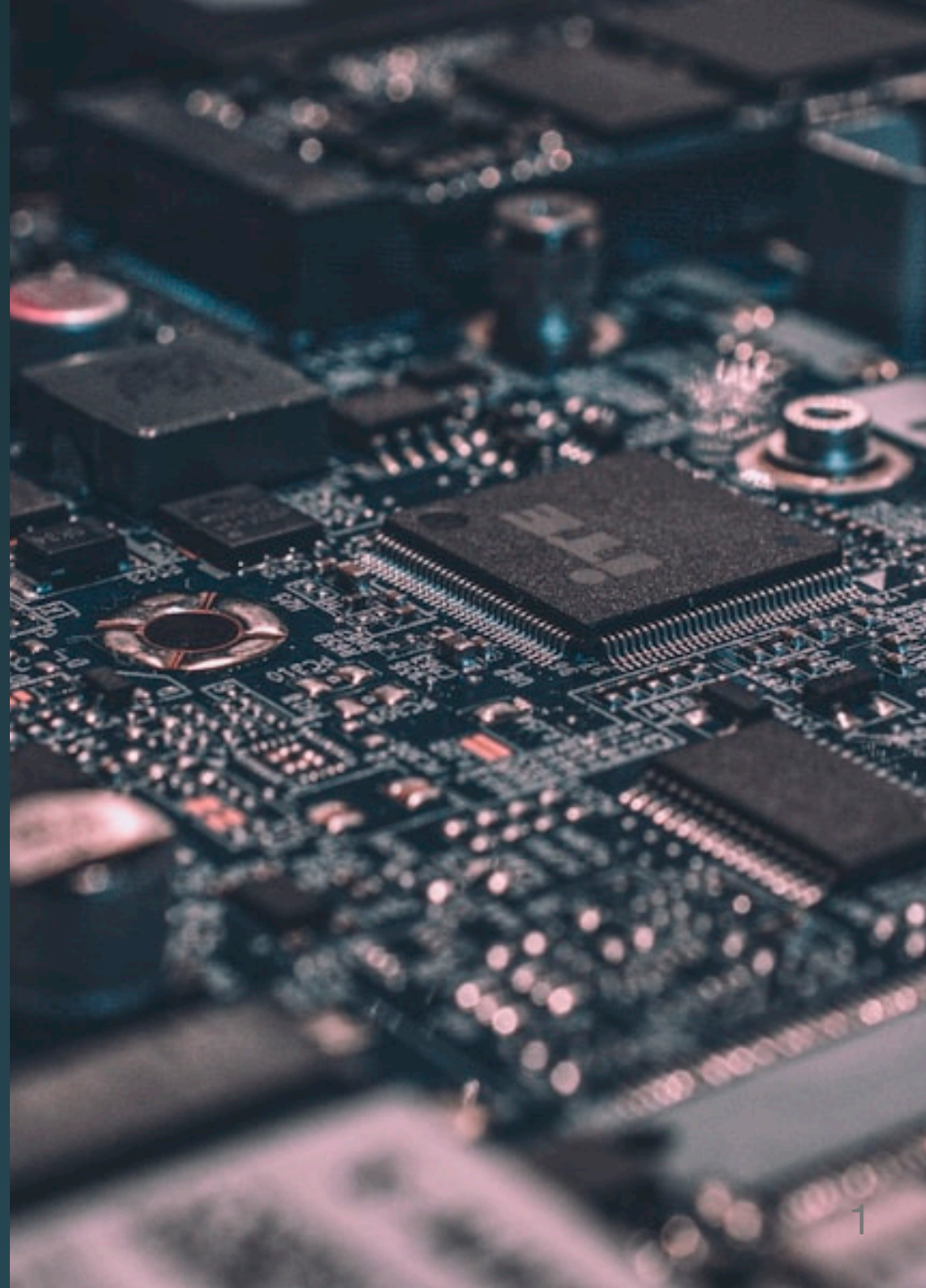


# Scrape Pipeline

Lesson 09 · ISBA 4715



# Agenda

|   | Part                     | What                                                                                  |
|---|--------------------------|---------------------------------------------------------------------------------------|
| 1 | <b>Scraping Concepts</b> | What it is, etiquette, the tools we'll use ( <i>slides</i> )                          |
| 2 | <b>Python Pipeline</b>   | Firecrawl search + scrape, saving to <code>knowledge/raw/</code> ( <i>live code</i> ) |
| 3 | <b>MCP Upgrade</b>       | Claude Code as the pipeline ( <i>demo</i> )                                           |
| 4 | <b>Your Project</b>      | Scrape at least one source into your repo                                             |

# How did Google build its first index?



Before anyone wrote an API for the web...



# How did Google build its first index?



Before anyone wrote an API for the web...

**Web scraping** — reading pages programmatically.





# Scraping is what you do when there is no API

Gathering data from websites **without an API** and **without a human clicking around in a browser**

# Scrape when the data has no other door



No API exists



The API is paywalled



Content changes constantly



Automate a human task

# Scrape like a guest, not a thief

| Rule                                      | Why                                               |
|-------------------------------------------|---------------------------------------------------|
| Check <code>robots.txt</code>             | Sites declare what bots can crawl                 |
| Rate limit —<br><code>time.sleep()</code> | Don't hammer the server                           |
| Respect Terms of Service                  | Legal + ethical obligation                        |
| Expect layout changes                     | HTML is not a stable contract                     |
| Be mindful of PII                         | Don't scrape what people did not consent to share |



**Scraped markdown feeds your  
knowledge base**



# Scraped markdown feeds your knowledge base

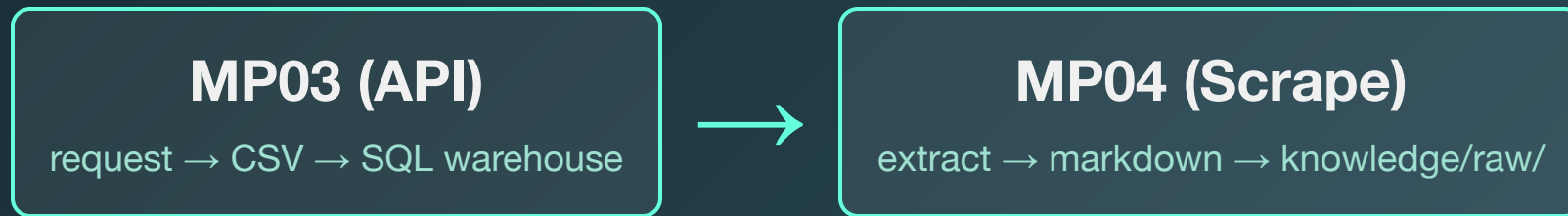
## MP03 (API)

request → CSV → SQL warehouse

# Scraped markdown feeds your knowledge base



# Scraped markdown feeds your knowledge base



These feed your **knowledge base** — the unstructured side of your portfolio project.

# You are building a RAG system

**R**etrieval-**A**ugmented **G**eneration — the standard pattern for AI systems that answer questions using your own content rather than just what a model memorized during training.

Every knowledge-base chatbot, internal-docs search, and "chat with your data" product you have seen is RAG.

# LLMs made stuff up. RAG fixed that.



LLMs on their own confabulate. RAG grounds answers in documents you supply. Your project is the agentic flavor — Claude Code chooses which files in `knowledge/raw/` to read for each question.

# You already have the four RAG parts



**Retrieval** — Firecrawl searches the web and pulls back markdown

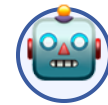


**Document store** —

`knowledge/raw/` holds the scraped markdown



**Indexed notes** — `knowledge/wiki/`  
(you build this in Milestone 02)



**Generation** — Claude Code reads the notes and answers grounded in them

No vector database, no embeddings — Claude Code uses grep, file reads, and its own reasoning. Simple RAG beats complex RAG until you have a reason otherwise.



# One API, Search + Extract



# JSON for scripts. Prose for humans.



vs.



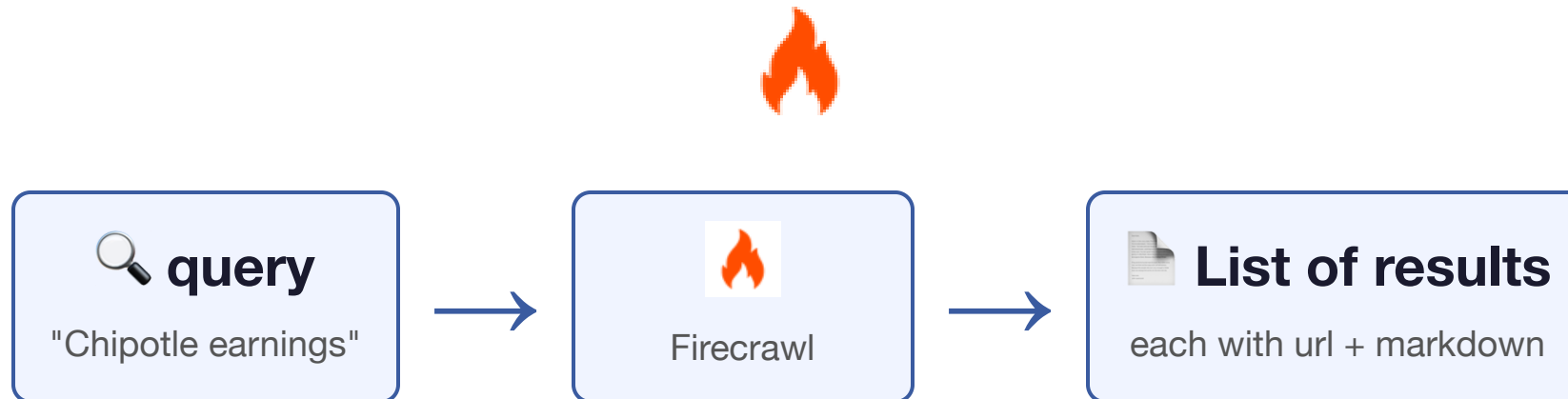
| Firecrawl (API)                                  | WebSearch (Claude Code tool)         |
|--------------------------------------------------|--------------------------------------|
| Returns structured data (URLs, titles, markdown) | Returns prose for the agent to read  |
| Designed for programmatic pipelines              | Built into Claude Code conversations |
| Same response schema every call                  | Different synthesized text each call |
| Full control: filters, domains, depth            | Whatever the tool decides            |

Cron jobs need structured data to parse. Humans need readable prose. Pick the right tool for the reader.

# We could use the SDK. We are using `requests` instead.

- Firecrawl publishes `firecrawl-py`
- You already know `requests` from MP03
- HTTP endpoints break less than SDK versions
- Same body, different wrapper

# One Firecrawl call = URLs + markdown



# BeautifulSoup still works. You do not need it.

- Parse HTML tag-by-tag
- Hunt for the right CSS selector
- Break every time the site redesigns



# BeautifulSoup still works. You do not need it.

- Parse HTML tag-by-tag
- Hunt for the right CSS selector
- Break every time the site redesigns

**This year we don't.** Firecrawl does all three in one call, and handles JavaScript rendering too. If you want BeautifulSoup for interview prep, read the docs — you do not need it for this project.

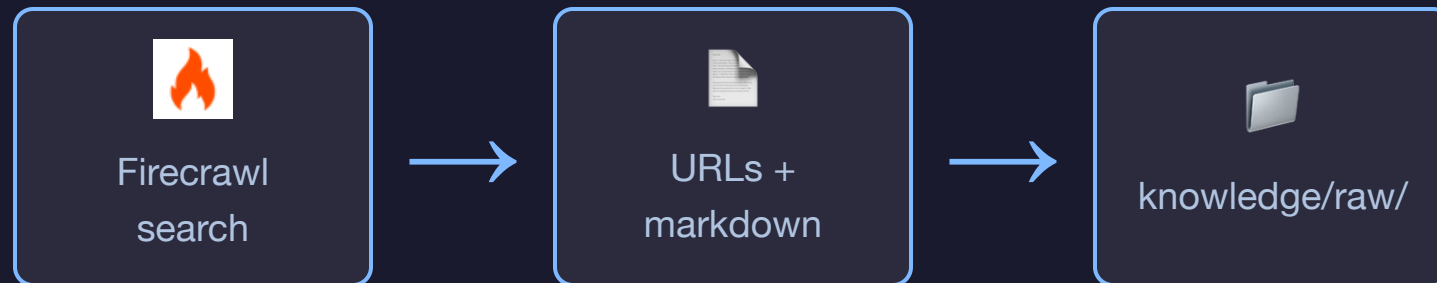


# Buy the API. Build the pipeline.

| Problem                                                                                                 | Don't                    | Do               |
|---------------------------------------------------------------------------------------------------------|--------------------------|------------------|
|  Find + scrape URLs    | Write a crawler + parser | <b>Firecrawl</b> |
|  Click through a login | Write a bot              | Playwright       |



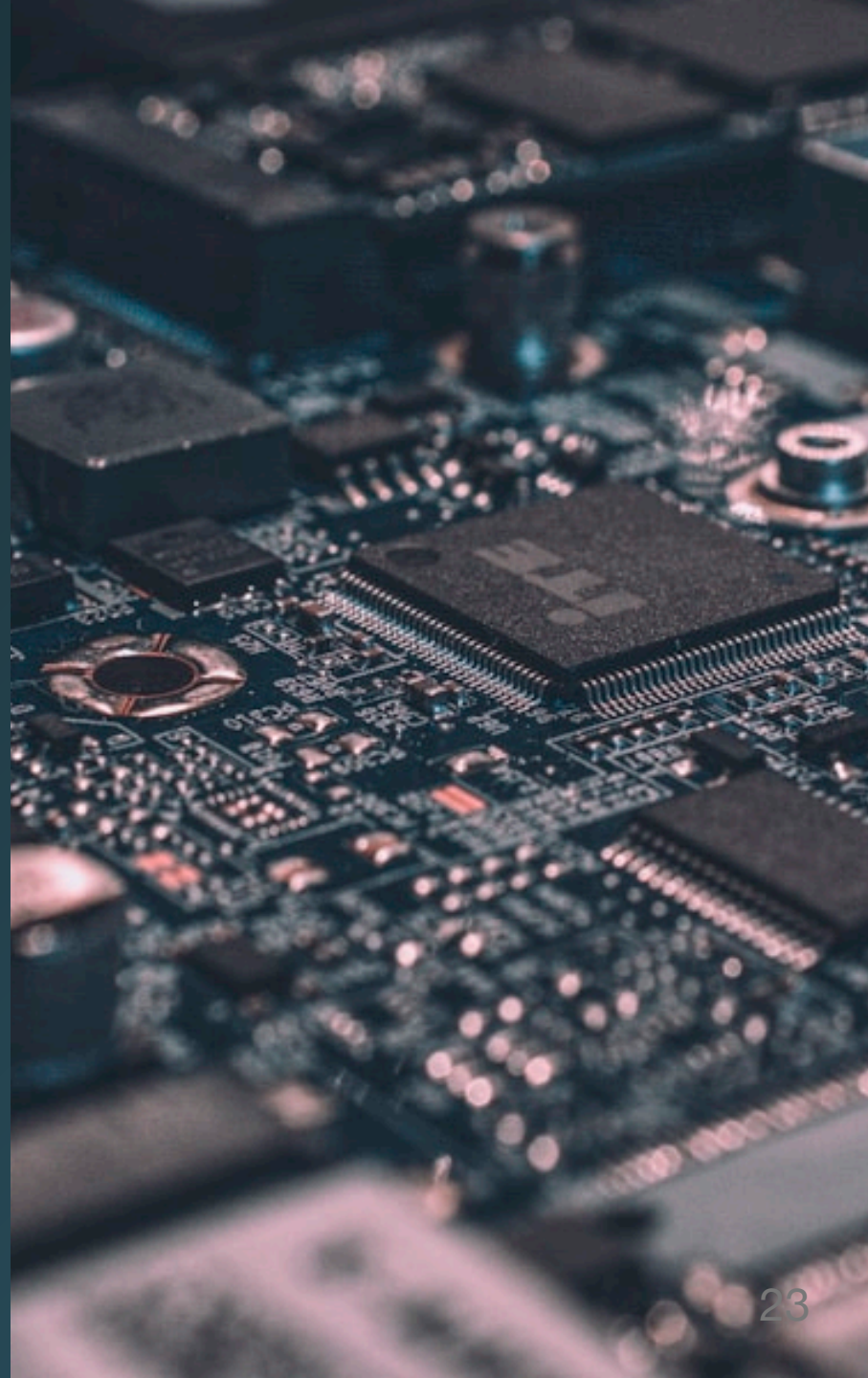
# Firecrawl finds. Firecrawl extracts. You save.



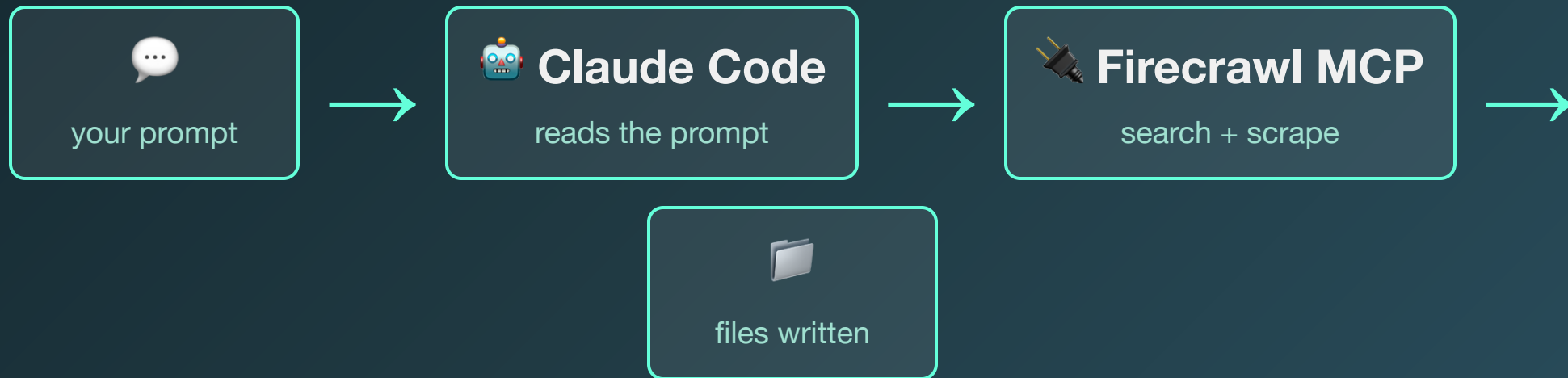
One API does the finding and the extracting. Your script saves the output.

# MCP turns your prompt into the pipeline

What if Claude Code could call Firecrawl **directly**, without you writing Python?



# MCP turns your prompt into the pipeline



**MCP servers** (Model Context Protocol) extend Claude Code with new tools. Install one, and Claude Code can use it inside any prompt.

# Python for production. MCP for exploration.

## Python (Part 02)

- ~30 lines of requests.post, loops, file writes
- Version-controlled
- Runs in GitHub Actions

## MCP (Part 03)

"Find 5 URLs about Chipotle's recent earnings and save each as markdown in  
`knowledge/raw/`."

**Same result. No code.**

# Chipotle IR: public, rich, scrape-friendly



**IR** = Investor Relations



Legally required to be public



Press releases, bios, earnings



Great knowledge-base fodder

# One account, 20,000 credits with STUDENTEDU



| Service                | URL                                                                               | Free Tier                                          |
|------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------|
| Firecrawl              | <a href="https://firecrawl.dev">firecrawl.dev</a>                                 | 500 credits                                        |
| <b>Student Program</b> | <a href="https://firecrawl.dev/student-program">firecrawl.dev/student-program</a> | <b>20,000 credits</b> with <code>.edu</code> email |

**To redeem student credits:** Dashboard → **Settings** → **Billing** → apply coupon code `STUDENTEDU`.

**Sign up with your `.edu` email** (not GitHub OAuth) so the coupon can verify your school. No card required.

# Milestone 02 needs 15 sources. Start today.



**15 sources**, 3+ sites



Same pattern, any URL



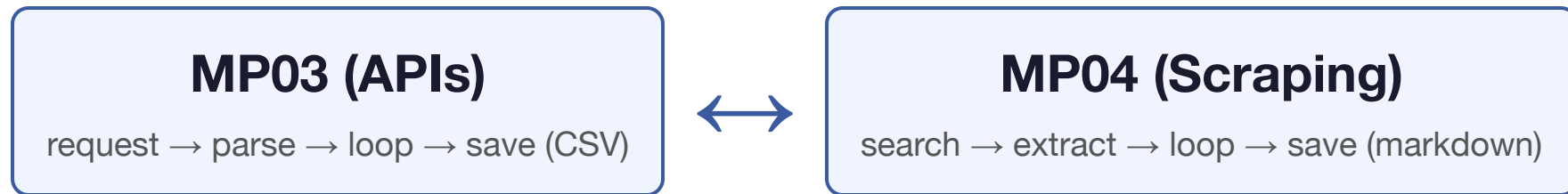
Job posting shapes content



Feeds your wiki pages



# Same pattern as MP03, different tool



Different tools. Same shape. If you learned MP03, you already know MP04.

**Next up:** code it together.